MathCAS v2

Getting started with numeric work

Development build, November 2026

This tutorial is for people who want numbers out of MathCAS: evaluating formulas to any number of digits, finding roots and minima, writing fast numeric loops and running simulations with random numbers. It assumes no earlier knowledge of MathCAS. Work through it at the REPL; every example was run on the current build and its output is copied unchanged.

Contents

1 Setting up **1**

2 Three kinds of numbers **2**

3 Machine reals and their limits **3**

4 Arbitrary precision with balls **3**

5 Complex numbers **4**

6 Exact first, numbers last **4**

7 Iterations **5**

8 Finding roots **5**

9 Finding minima and maxima **6**

10 Numeric integration **7**

11 Differential equations **8**

12 Fast numeric loops: the compiler **9**

13 Random numbers and simulation **10**

14 Failures, limits and interrupts **11**

15 Quick reference **11**

16 Coming next **11**

1 Setting up

The distribution folder (or the installed program) holds `mc-repl.exe`, the interactive REPL, which also runs scripts and notebooks; `mc-kernel.exe` and `mc-lsp.exe` for editors; `mc-bench.exe`, the

benchmark harness; `examples\` and the VS Code extension. Start the REPL and type an expression; its value is printed below it.

```
mc> 1/3 + 1/6
1/2
mc> exp(1.5)
4.4816890703380645
```

Useful REPL commands while doing numerics:

`:time`	toggle timing: prints `time: ... ms` after each result
`:load file.mc`	read and run a script
`:history`, `:redo [k]`	show earlier inputs, run one again
Ctrl+C	stop a long computation and keep the session

Longer work goes into a script file `work.mc`, run with `mc -repl work.mc` (add `-i` to stay in the REPL afterwards). `say(a, b, ...)` prints its arguments from a script.

Notebooks. Every example in this tutorial also runs in a Jupyter notebook, in Jupyter itself or in VS Code. With Python and `ipykernel` installed, the installer registers the kernel (or run `python jupyter\install.py` in the program folder). Then open a `.ipynb` file, choose the kernel **MathCAS v2**, and each cell shows the value of its last statement. Interrupt stops a running cell; restart gives a fresh session. `examples\jupyter.ipynb` is a sample.

2 Three kinds of numbers

MathCAS keeps three kinds of numbers apart, and knowing which one you hold is the key to numeric work.

Kind	Written	Behaviour
exact	`3`, `2/7`, `sqrt(2)`, `pi`	integers and rationals of any size; constants and roots stay symbolic
machine real	`1.5`, `2.0e-8`, `n(x)`	IEEE double precision: fast, about 16 digits, no error tracking
ball	`1.25~30`, `n(x, 30)`	a value with a guaranteed error bound, at any precision

Exact arithmetic never rounds. `n` turns an exact quantity into a number: with one argument into a machine real, with a digit count into a ball.

```
mc> q = 1/3 + 2/7
mc> q
13/21
mc> n(q)
0.6190476190476191
mc> n(q, 40)
0.6190476190476190476190476190476190476190476
```

Mixing kinds gives the least exact kind present: exact < ball < machine real. A single machine real in a computation makes the whole result a machine real, so ask for digits at the start, not at the end:

```
mc> n(pi, 30) + 1/2
3.641592653589793238462643383279503
mc> n(pi, 30) + 0.5
3.641592653589793
```

`precision(x)` tells you what you hold: the number of correct decimal digits, `inf` for an exact number, about 16 for a machine real.

```
mc> precision(1/3)
inf
mc> precision(1.5)
15.954589770191003
mc> precision(n(pi, 30))
34.91947949702182
```

3 Machine reals and their limits

Machine reals are what most numeric software uses. They are fast, but they round every operation, silently:

```
mc> 0.1 + 0.2
0.30000000000000004
mc> 1.0e30 + 1 - 1.0e30
0.0
mc> 10^30 + 1 - 10^30
1
```

The second line lost the `1` entirely: next to 10^{30} it falls below the 16 digits a machine real carries. The exact computation on the third line has no such problem. This is the usual pattern in MathCAS: *compute exactly while you can, turn into numbers as late as possible.*

4 Arbitrary precision with balls

`n(x, d)` evaluates `x` with at least `d` correct digits. The result is a *ball*: a midpoint and a radius that is guaranteed to enclose the true value. MathCAS prints only the digits that the radius guarantees.

```
mc> n(pi, 50)
3.14159265358979323846264338327950288419716939937510 5821
mc> n(sqrt(2), 40)
1.41421356237309504880168872420969807856967 19
mc> n(exp(1), 30)
2.71828182845904523536028747135266 25
mc> n(euler_gamma, 30)
0.57721566490153286060651209008240 24
```

You can also write a ball directly: `1.25~20` is 1.25 with 20 correct digits.

Balls track their own error. When digits are lost, the ball knows, and prints fewer:

```
mc> b = 1~20
mc> (b + 10^-15) - b
1.000e-15
mc> n(sqrt(2), 20)^2 - 2
ball(-2.0679515313825691872e-25, 4.08e-25)
```

Starting from 20 digits, the subtraction leaves only four. The second result is a number so close to zero that not a single digit is certain, so MathCAS shows the ball as it is: a midpoint and a radius. A ball whose interval contains zero is the honest answer to "is this zero?": as far as 20 digits can tell.

Tip. When a result has fewer digits than you asked for, the computation lost precision on the way (a cancellation like the one above). Ask for more digits at the start: `n(expr, 50)` instead of `n(expr, 20)`.

Functions of balls. Every elementary function (`exp`, `log`, `sin` ... `tanh`, `sqrt`, powers) evaluates on balls to any precision. Large arguments are no problem, because the argument reduction is done with enough digits:

```
mc> n(sin(10^20), 20)
-0.64525128526578084420581 2
mc> n(exp(100), 20)
2.68811714181613544841263e43
```

Working with the digits. A few helpers for the boundary between numbers and exact values:

```
mc> rationalize(0.75)
3/4
mc> rationalize(n(pi), 1/1000)
201/64
mc> set_precision(0.1, 30)
0.100000000000000005551115123126
mc> chop(n(exp(pi*1im)))
-1.0
```

`set_precision` shows that the machine real `0.1` is not one tenth: it is the nearest binary fraction, and with 30 digits you see the difference.

5 Complex numbers

The imaginary unit is written `im`, as in `3 + 4im`. Machine complex numbers and complex balls print as `complex(re, im)`.

```
mc> abs(3 + 4im)
5
mc> n(exp(1 + 2im))
complex(-1.1312043837568135, 2.4717266720048188)
mc> n(exp(1 + 2im), 30)
complex(-1.13120438375681363843125525555107947, 2.47172667200481892761693089355 1665)
mc> n(sqrt(-2))
complex(0.0, 1.4142135623730951)
```

`re`, `im_part`, `conjugate` and `arg` take complex numbers apart.

6 Exact first, numbers last

Many questions have an exact answer that MathCAS can find; you then evaluate it to as many digits as you need. Differentiation, integration and series are symbolic:

```
mc> d(sin(x)^2, x)
2*cos(x)*sin(x)
mc> integrate(1/(1 + x^2), [x, 0, 1])
(1/4)*pi
mc> n(integrate(1/(1 + x^2), [x, 0, 1]), 30)
0.785398163397448309615660845819875 7
mc> series(exp(x), x, 0, 5)
1 + x + x^2/2 + x^3/6 + x^4/24 + x^5/120 + O(x^6)
```

The same holds for sums: add exactly, then evaluate.

```
mc> n(sum(map(k => 1/k^2, range(1000))), 30)
1.643934566681559803139058023822215 6
```

Lists of numbers come from `range`, `table`, `map` and `nest_list`; `n` works on whole lists.

```
mc> n(range(0, 1, 1/4))
[0.0, 0.25, 0.5, 0.75, 1.0]
mc> table(n(k*pi/4), k, range(0, 4))
[0.0, 0.7853981633974483, 1.5707963267948966, 2.356194490192345, 3.141592653589793]
```

Arithmetic works element by element on lists, so a whole vector of values is one expression:

```
mc> [1, 2, 3] + [10, 20, 30]
[11, 22, 33]
mc> n(pi)*[1, 2, 3]/2
[1.5707963267948966, 3.141592653589793, 4.71238898038469]
mc> [1, 2, 3]^2
[1, 4, 9]
```

7 Iterations

`nest_list(f, x, k)` applies `f` to `x` `k` times and keeps every step: a fixed-point iteration in one line. Newton's method for $\sqrt{2}$, $x \mapsto (x + 2/x)/2$, doubles the number of correct digits each step:

```
mc> nest_list(x => (x + 2/x)/2, 1.0, 5)
[1.0, 1.5, 1.4166666666666665, 1.4142156862745097, 1.4142135623746899,
    1.414213562373095]
```

The same iteration on a 30-digit ball needs only one more step to reach 30 digits, and the result says how many it has:

```
mc> nest(x => (x + 2/x)/2, 1~30, 6)
1.41421356237309504880168872421
```

Symbolic and numeric work combine naturally: here is Newton's method for any function, with the derivative computed symbolically once.

```
fn newton(f, x0, steps) = {
  var x = x0
  let df = d(f(t), t)          # the derivative, exactly
  for i in range(steps) { x = x - f(x)/rewrite(df, [t -> x]) }
  x
}
```

```
mc> newton(t => t^2 - 2, 1.0, 6)
1.414213562373095
mc> newton(t => t^2 - 2, 1~50, 8)
1.41421356237309504880168872420969807856967187538
```

Of course you rarely need to write this yourself: that is what `find_root` is for.

8 Finding roots

`find_root(f, [x, x0])` finds a root of `f` (or a solution of `lhs == rhs`) near the starting value `x0`. It uses Newton's method with the exact derivative. The result is a list of rules, so you can substitute it into other expressions.

```
mc> find_root(cos(x) - x, [x, 1])
[x -> 0.7390851332151607]
mc> r = find_root(cos(x) - x, [x, 1])
mc> rewrite(x, r)
0.7390851332151607
```

The second argument chooses the method:

`[x, x0]`	Newton from `x0` (exact derivative; secant steps when there is none)
`[x, x0, x1]`	secant method from two starting values
`[x, x0, a, b]`	Brent's method inside the bracket $[a, b]$ (a sign change is required)
`[[x, x0], [y, y0]]`	a system of equations: Newton with the exact Jacobian, Broyden as a fallback

```
mc> find_root(x^3 - 2x - 5, [x, 2, 3])
[x -> 2.0945514815423265]
mc> find_root([x^2 + y^2 - 4, x - y], [[x, 1], [y, 1]])
[x -> 1.4142135623730951, y -> 1.4142135623730951]
mc> find_root(x^2 + 1, [x, 1 + 1im])
[x -> complex(0.0, 1.0)]
```

A complex starting value searches in the complex plane, as on the last line.

Many digits. The option `digits: d` refines the machine root with balls. For a simple root of a real function in one variable, the result is *verified*: the ball is proved to contain exactly one root.

```
mc> find_root(cos(x) == x, [x, 1], digits: 40)
[x -> 0.739085133215160641655312087673873404013 4]
mc> find_root(sin(x), [x, 3], digits: 60)
[x -> 3.14159265358979323846264338327950288419716939937510582097 49]
```

When it fails. A root finder that cannot converge does not stop your session. It returns a `failure` value with its best estimate and an error estimate:

```
mc> find_root(exp(x) + 1, [x, 0])
failure(not_converged, find_root, [x -> -8136.132507311752], 8125.743451212822)
```

($e^x + 1$ has no real root; Newton's method ran off to $-\infty$.) Failures are ordinary values: test them with `is_failure`, and choose a different starting value or a bracket.

9 Finding minima and maxima

`find_minimum(f, [x, x0])` returns the minimum value and where it is attained; `find_maximum` works the same way. Several variables use BFGS with the exact gradient.

```
mc> find_minimum(x^4 - 3x^2 + x, [x, 1])
[-1.0702301817761541, [x -> 1.1309011226299859]]
mc> find_maximum(sin(x)*exp(-x/5), [x, 1])
[0.7450600126415114, [x -> 1.373400766945016]]
mc> find_minimum((x - 1)^2 + 100(y - x^2)^2, [[x, -1], [y, 1]])
[0.0, [x -> 1.0, y -> 1.0]]
```

The last one is Rosenbrock's banana function, a classic hard case: a long curved valley with its minimum at $(1, 1)$.

A precision subtlety. Near a minimum a function is flat, so its value changes very little when the position does. A minimiser can therefore locate the *value* to full precision but the *position* only to about half the digits:

```
mc> f = x^3 - 6x^2 + 9x + 1
mc> find_minimum(f, [x, 2.5])
[0.9999999999999929, [x -> 3.0000000298023224]]
mc> find_root(d(f, x), [x, 2.5])
[x -> 3.0]
```

When you need the position accurately, solve $f'(x) = 0$ with `find_root`, as on the last line.

10 Numeric integration

Many integrals have no closed form that `integrate` can find; it then leaves them as they are. `n_integrate(f, [x, a, b])` computes their value numerically.

```
mc> integrate(exp(-x^2), [x, 0, 1])
integrate(exp(-x^2), [x, 0, 1])
mc> n_integrate(exp(-x^2), [x, 0, 1])
0.7468241328124272
mc> n_integrate(exp(-x^2), [x, -inf, inf])
1.7724538509055163
```

(The last one is $\sqrt{\pi}$.) By default the integrand is compiled, the interval is split adaptively where the integrand is hard (Gauss–Kronrod rules), and the result is good to about eight digits relative, usually to all sixteen. Singularities at the ends are handled by a second method (double-exponential rules), chosen automatically:

```
mc> n_integrate(1/sqrt(x), [x, 0, 1])
1.9999999999999998
mc> n_integrate(log(x)^2, [x, 0, 1])
2.0
```

Ranges. `inf` and `-inf` are allowed as limits. List known trouble spots inside the range between the limits, `[x, a, s1, s2, b]`, to split there. More ranges give a multiple integral; inner limits may depend on outer variables:

```
mc> n_integrate(1/(1 + x^2), [x, -1, 0, 1])
1.5707963267948966
mc> n_integrate(x*y, [x, 0, 1], [y, 0, x])
0.12500000000000003
```

Many digits, and guarantees. `digits: d` integrates on balls to `d` digits. The error is then estimated, not proved; add `rigorous: true` for a ball that is guaranteed to contain the integral (for integrands built from the elementary functions, on a finite range).

```
mc> n_integrate(exp(-x^2), [x, 0, 1], digits: 40)
0.7468241328124270253994674361318530053545000
mc> n_integrate(exp(-x^2), [x, 0, 1], digits: 30, rigorous: true)
0.746824132812427025399467436131850
```

When it fails. A divergent integral or one that cannot reach the goal gives a failure with the estimate and the error estimate, as `find_root` does:

```
mc> n_integrate(1/x, [x, 0, 1])
failure(not_converged, n_integrate, 44.501326166534795, 13.112269516493784)
```

Other options: `accuracy_goal` and `precision_goal` (in digits); `method`, either `gauss_kronrod` or `double_exponential`; `max_subdivisions`; and `compile: false`.

11 Differential equations

`n_dsolve(equations, unknowns, [t, t0, t1])` solves ordinary differential equations numerically. Write the equations with primes and give the initial conditions as equations too. The result is a list of rules, like `find_root`'s: each unknown becomes an `interpolating_function`, a function you can evaluate anywhere in the range.

```
mc> sol = n_dsolve([y'(t) == -y(t)/2, y(0) == 1], y, [t, 0, 10])
mc> sol
[y -> interpolating_function("y", [0.0, 10.0], 39, "66d23807a7863f0f")]
mc> rewrite(y(2), sol)
0.3678794417481203
mc> rewrite(y([0, 1, 2]), sol)
[1.0, 0.6065306598001193, 0.3678794417481203]
```

The summary shows the range and the number of steps taken (39). The exact value e^{-1} is 0.36787944117144233: by default the solution is accurate to about eight digits; ask for more with `precision_goal: 12`. Evaluating outside the range gives `failure(out_of_range, ...)`.

Higher order and systems. Second and higher derivatives are written with more primes; systems list their unknowns. Products must be written with `*` (`x(t)*y(t)`: `x(t) y(t)` or `x(t)(28 - z(t))` would read as a call). The Lorenz system:

```
lor = n_dsolve([x'(t) == 10*(y(t) - x(t)),
                y'(t) == x(t)*(28 - z(t)) - y(t),
                z'(t) == x(t)*y(t) - 8*z(t)/3,
                x(0) == 1, y(0) == 1, z(0) == 1], [x, y, z], [t, 0, 20])
```

```
mc> rewrite([x(20), y(20), z(20)], lor)
[13.804638918840993, 12.968730181543803, 34.912416415052746]
```

Stiff equations. Some equations mix very fast and very slow behaviour; explicit methods then crawl. `n_dsolve` notices this and switches to an implicit method (`method: stiff` forces it). The Van der Pol oscillator with $\mu = 1000$ is a classic test:

```
mc> vdp = n_dsolve([v''(t) == 1000*(1 - v(t)^2)*v'(t) - v(t), v(0) == 2, v'(0) == 0],
    v, [t, 0, 3000])
mc> vdp
[v -> interpolating_function("v", [0.0, 3000.0], 2920, "8caf0157fb22950b")]
```

That took about 5 ms. The right-hand side is compiled, as for `n_integrate`.

Events. `events: [g == 0]` stops the integration where `g` crosses zero: here, when a dropped ball reaches the ground. The rules then also say where it stopped.

```
mc> fall = n_dsolve([h''(t) == -9.81, h(0) == 10, h'(0) == 0], h, [t, 0, 5], events:
    [h(t) == 0])
mc> fall
```

```
[h -> interpolating_function("h", [0.0, 1.4278431229270643], 4, "196ff5f967e63e16"),
    events -> [[1.4278431229270643, 1]]]
```

One value, many digits. `n_dsolve_value(equations, expr, range)` gives the value of an expression of the solution directly. With `digits: d` it switches to a Taylor-series method on balls. The equation $y' = 1 + y^2$, $y(0) = 0$ has the solution $\tan t$:

```
mc> n_dsolve_value([y'(t) == 1 + y(t)^2, y(0) == 0], y(3/2), [t, 0, 3/2], digits: 30)
14.101419947171719387646083651 99
mc> n(tan(3/2), 30)
14.101419947171719387646083651 98776
```

12 Fast numeric loops: the compiler

Loops written in MathCAS are interpreted, which costs about a microsecond per operation. For numeric loops MathCAS has a compiler that produces machine code. A function qualifies when its parameters are typed (`Int` or `Real`) and its body uses arithmetic, comparisons, `if`, `let`/`var`, `while`, `for` over ranges and the elementary functions. Put `@compiled` before `fn`, or call `compile(f)` later.

```
fn slow(n: Int) = { var s = 0.0; for k in range(n) { s = s + sin(1.0*k) }; s }
```

```
mc> :time
timing on
mc> slow(200000)
-0.03338662721108862
time: 1.143 s
mc> compile(slow)
slow
mc> slow(200000)
-0.03338662721108862
time: 1.136 ms
```

A thousand times faster, with the same result to the last bit. That is a guarantee, not luck: whenever compiled code meets something it cannot do natively (an integer overflow, the square root of a negative number, a function it does not know), the call falls back to the interpreter. Compiling never changes an answer.

Two more examples: the trapezoidal rule for $\int_0^1 e^{-x^2}\,dx$, and the logistic map, a classic of chaos.

```
@compiled fn trap(m: Int) = {
  let h = 1.0/m
  var s = 0.5*(exp(0.0) + exp(-1.0))
  for k in range(m - 1) { s = s + exp(-(k*h)^2) }
  s*h
}
@compiled fn logistic(r: Real, n: Int) = {
  var x = 0.5
  for k in range(n) { x = r*x*(1.0 - x) }
  x
}
```

```
mc> trap(1000)
0.7468240714991843
mc> logistic(3.7, 1000)
0.8101078985832381
```

`compiled_info(f)` shows what the compiler made of a function: parameter and result types, code size, compile time, and how many calls ran compiled or fell back.

```
mc> compiled_info(trap)
[params -> [Int], result -> Real, hir_nodes -> 41, lir_instructions -> 73, code_bytes
    -> 856, back_end -> "cranelift", compile_microseconds -> 1591, inlined -> [], state
    -> "active", runs -> 1, declines -> 0]
```

Tip. Compile the inner loop, not the whole program. Exact arithmetic, symbolic work and balls stay in the interpreter, where they belong: at high precision the arithmetic itself, not the interpreter, takes the time.

13 Random numbers and simulation

The session has one random generator. `seed(n)` makes a run reproducible.

`random_real()`, `random_real([a, b], n)`	uniform reals in $[0, 1)$ or $[a, b)$; with `n`, a list
`random_integer([a, b], n)`	uniform integers from `a` to `b`
`random_normal(mu, sigma)`	normally distributed reals
`random_choice(xs)`, `random_sample(xs, n)`	elements of a list, with and without repetition

```
mc> seed(42)
mc> random_real()
0.3354948980859701
mc> random_real([0, 10], 3)
[1.5456113417635597, 9.64328247450341, 4.275784977896785]
mc> random_integer([1, 6], 5)
[3, 6, 5, 6, 3]
```

A Monte Carlo estimate of π: the fraction of random points in the unit square that fall inside the quarter circle is $\pi/4$.

```
mc> seed(7)
mc> 4.0*count_if(range(100000), k => random_real()^2 + random_real()^2 < 1)/100000
3.14604
```

The error of such an estimate shrinks like $1/\sqrt{N}$: a hundred times more points for one more digit. So simulations want speed: compiled functions can draw random numbers, and draw exactly the numbers the interpreter would.

```
@compiled fn mc_pi(k: Int) = {
  var hits = 0
  for i in range(k) {
    let a = random_real()
    let b = random_real()
    if a*a + b*b < 1.0 then hits = hits + 1 else hits
  }
  4.0*hits/k
}
```

```
mc> seed(7)
mc> mc_pi(10^7)
3.140728
```

Ten million points take about a quarter of a second. Compiled code also handles complex numbers (`complex(x, y)`, `im`, arithmetic, `abs`, `exp` and the other elementary functions): a Mandelbrot escape-time loop compiles.

14 Failures, limits and interrupts

Numeric work fails in ordinary ways: an iteration does not converge, a value is not a number, a computation takes too long. MathCAS reports all of these as values instead of crashing or hanging:

- `failure(not_converged, f, estimate, error)`: a solver gave up; the estimate is often still useful.
- `failure(not_numeric, f, point)`: the function did not give a number at some point (a typo in a variable name is the usual cause).
- `failure(division_by_zero)`, `failure(argument, f, k)`: bad input.
- **Ctrl+C** interrupts any computation, compiled loops included, and keeps the session and its definitions.

`is_failure(v)` and `failure_tag(v)` let a script react to them.

15 Quick reference

Write	Meaning
`n(x)`, `n(x, d)`	machine real; ball with at least `d` correct digits
`1.25~30`	a ball literal with 30 digits
`precision(x)`, `accuracy(x)`	correct digits: relative, absolute
`set_precision(x, d)`, `rationalize(x, tol)`, `chop(x)`	change precision, back to exact, drop tiny parts
`pi`, `e`, `euler_gamma`, `catalan`, `im`	constants
`re`, `im_part`, `abs`, `arg`, `conjugate`	complex parts
`floor`, `ceiling`, `round`, `clip`, `rescale`, `sign`	rounding and ranges
`d(f, x)`, `integrate(f, x)`, `integrate(f, [x, a, b])`	exact derivative, antiderivative, definite integral
`series(f, x, x0, k)`	power series to order `k`
`find_root(f, [x, x0])` and variants	roots (Newton, secant, Brent, systems, complex)
`find_minimum`, `find_maximum`	local extrema (Brent, BFGS)
`n_integrate(f, [x, a, b])`	numeric integral (also infinite and multiple ranges)
`n_dsolve(eqs, y, [t, t0, t1])`, `n_dsolve_value`	differential equations
`nest(f, x, k)`, `nest_list(f, x, k)`	iterate a function
`range`, `table`, `map`, `sum`, `total`	build and reduce lists
`@compiled fn`, `compile(f)`, `compiled_info(f)`	compile to machine code, inspect
`seed`, `random_real`, `random_integer`, `random_normal`	random numbers
`option: value` after the arguments	options, e.g. `digits: 40`

16 Coming next

Packed numeric arrays (long lists of numbers stored compactly and computed on directly), plots, and then special functions (`gamma`, `zeta`, Bessel functions), linear algebra and equation solving.